

```

options(OutDec = ".") # Asegurar punto decimal en este chunk

# Establecer semilla aleatoria
set.seed(sample(1:10000, 1))

# Aleatorizar dimensiones del parabrisas
largo_base <- sample(10:15, 1) # Largo base entre 10 y 15 cm
ancho_base <- sample(6:10, 1)  # Ancho base entre 6 y 10 cm

# Aleatorizar longitud de las plumillas
longitud_plumilla_base <- sample(4:8, 1) # Longitud base entre 4 y 8 cm

# Aleatorizar términos para el contexto del problema
objetos <- c("parabrisas", "ventana frontal", "vidrio delantero", "cristal frontal")
objeto <- sample(objetos, 1)

vehiculos <- c("carrito de juguete", "coche en miniatura", "automóvil de juguete",
              "vehículo de juguete", "modelo a escala")
vehiculo <- sample(vehiculos, 1)

terminos_limpiador <- c("plumilla", "limpiador", "limpiaparabrisas", "escobilla")
termino_limpiador <- sample(terminos_limpiador, 1)

formas <- c("rectangular", "de forma rectangular", "con forma de rectángulo")
forma <- sample(formas, 1)

angulos <- c("90°", "90 grados", "ángulo recto")
angulo <- sample(angulos, 1)

# Aleatorizar la opción correcta (C es la correcta en la imagen original)
opciones <- c("A", "B", "C", "D")
opcion_correcta <- "C"
indice_correcto <- which(opciones == opcion_correcta)

# Vector de solución para r-exams
solucion <- integer(4)
solucion[indice_correcto] <- 1

# Configurar opciones incorrectas con variaciones específicas
# En estas variables definimos las dimensiones para cada opción
# Opciones originales de las imágenes:
# A: largo=13, ancho=8, plumilla1=6, plumilla2=6
# B: largo=13, ancho=8, plumilla1=6, plumilla2=8
# C: largo=13, ancho=8, plumilla1=6, plumilla2=6 (CORRECTA)
# D: largo=13, ancho=6, plumilla1=8, plumilla2=8

# Mantener las proporciones pero con las dimensiones aleatorizadas
opcion_A <- list(
  largo = largo_base,
  ancho = ancho_base,
  plumilla1 = longitud_plumilla_base,
  plumilla2 = longitud_plumilla_base
)

```

```

opcion_B <- list(
  largo = largo_base,
  ancho = ancho_base,
  plumilla1 = longitud_plumilla_base,
  plumilla2 = longitud_plumilla_base + sample(1:3, 1)
)

opcion_C <- list(
  largo = largo_base,
  ancho = ancho_base,
  plumilla1 = longitud_plumilla_base,
  plumilla2 = longitud_plumilla_base
)

opcion_D <- list(
  largo = largo_base,
  ancho = ancho_base - sample(1:2, 1),
  plumilla1 = longitud_plumilla_base + sample(1:3, 1),
  plumilla2 = longitud_plumilla_base + sample(1:3, 1)
)

# Las dimensiones de la respuesta correcta
largo <- opcion_C$largo
ancho <- opcion_C$ancho
longitud_plumilla <- opcion_C$plumilla1

# Para mostrar en el enunciado
largo_texto <- largo
ancho_texto <- ancho
longitud_plumilla_texto <- longitud_plumilla

options(OutDec = ".") # Asegurar punto decimal en este chunk

# Función para generar código Python para dibujar el esquema
generar_esquema_python <- function(opcion, letra) {
  codigo <- paste0('
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import numpy as np
from matplotlib.path import Path

# Crear figura y ejes
plt.figure(figsize=(5, 4))
ax = plt.gca()

# Dimensiones
largo = ', opcion$largo, '
ancho = ', opcion$ancho, '
plumilla1 = ', opcion$plumilla1, '
plumilla2 = ', opcion$plumilla2, '

# Dibujar el rectángulo del parabrisas
parabrisas = patches.Rectangle((0, 0), largo, ancho, linewidth=1, edgecolor="black", facecolor="none")

```

```

ax.add_patch(parabrisas)

# Letra identificadora
ax.text(-0.5, ancho/2, "'", letra, "'", fontsize=12, bbox=dict(facecolor="skyblue", alpha=0.5))

# Configurar colores
color_plumilla = "brown"
color_dimension = "red"

# Dibujar arcos de las plumillas
# Primera plumilla
theta = np.linspace(0, np.pi/2, 100)
x1 = plumilla1 * np.cos(theta)
y1 = plumilla1 * np.sin(theta)
ax.plot(x1, y1, color=color_plumilla, linewidth=1)

# Línea de la primera plumilla
ax.plot([0, plumilla1], [0, 0], color=color_plumilla, linewidth=1)

# Marca de ángulo recto primera plumilla
ax.plot([1, 1, 0], [0, 1, 1], color=color_plumilla, linestyle=":", linewidth=0.5)

# Segunda plumilla
theta = np.linspace(0, np.pi/2, 100)
x2 = plumilla1 + plumilla2 * np.cos(theta)
y2 = plumilla2 * np.sin(theta)
ax.plot(x2, y2, color=color_plumilla, linewidth=1)

# Línea de la segunda plumilla
ax.plot([plumilla1, plumilla1 + plumilla2], [0, 0], color=color_plumilla, linewidth=1)

# Marca de ángulo recto segunda plumilla
ax.plot([plumilla1 + 1, plumilla1 + 1, plumilla1], [0, 1, 1], color=color_plumilla, linestyle=":", linewidth=0.5)

# Dimensiones del parabrisas
# Largo
ax.annotate("", xy=(0, ancho + 0.8), xytext=(largo, ancho + 0.8),
            arrowprops=dict(arrowstyle="<->", color=color_dimension))
ax.text(largo/2, ancho + 1.2, f"{largo} cm", ha="center", color=color_dimension)

# Ancho
ax.annotate("", xy=(-0.8, 0), xytext=(-0.8, ancho),
            arrowprops=dict(arrowstyle="<->", color=color_dimension))
ax.text(-1.5, ancho/2, f"{ancho} cm", va="center", rotation=90, color=color_dimension)

# Dimensiones de las plumillas
# Plumilla 1
ax.annotate("", xy=(0, -0.8), xytext=(plumilla1, -0.8),
            arrowprops=dict(arrowstyle="<->", color=color_dimension))
ax.text(plumilla1/2, -1.2, f"{plumilla1} cm", ha="center", color=color_dimension)

# Plumilla 2
ax.annotate("", xy=(plumilla1, -0.8), xytext=(plumilla1 + plumilla2, -0.8),
            arrowprops=dict(arrowstyle="<->", color=color_dimension))

```

```

        arrowprops=dict(arrowstyle="<->", color=color_dimension))
ax.text(plumilla1 + plumilla2/2, -1.2, f"{plumilla2} cm", ha="center", color=color_dimension)

# Etiqueta de identificación para las plumillas
ax.text(plumilla1/2 - 0.5, plumilla1/2 + 1, "Plumilla", rotation=-45, ha="right", color=color_plumilla)
ax.text(plumilla1 + plumilla2/2 - 0.5, plumilla2/2 + 1, "Plumilla", rotation=-45, ha="right", color=col

# Configuración de ejes
ax.set_xlim(-2, largo + 2)
ax.set_ylim(-2, ancho + 2)
ax.set_aspect("equal")
ax.axis("off")

# Guardar figura
plt.tight_layout()
plt.savefig("opcion", letra, '.png', dpi=150, bbox_inches="tight")
plt.close()
')

    return(codigo)
}

# Generar código Python para cada opción
codigo_opcion_A <- generar_esquema_python(opcion_A, "A")
codigo_opcion_B <- generar_esquema_python(opcion_B, "B")
codigo_opcion_C <- generar_esquema_python(opcion_C, "C")
codigo_opcion_D <- generar_esquema_python(opcion_D, "D")

# Ejecutar código Python para generar las figuras
py_run_string(codigo_opcion_A)
py_run_string(codigo_opcion_B)
py_run_string(codigo_opcion_C)
py_run_string(codigo_opcion_D)

options(OutDec = ".") # Asegurar punto decimal en este chunk

# Código Python para generar el diagrama de plumillas con ángulo
codigo_diagrama_plumilla <- '
import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots(figsize=(8, 4))

# Rectángulo punteado
rect_x = [0.5, 6.7, 6.7, 0.5, 0.5]
rect_y = [0, 0, 3, 3, 0]
ax.plot(rect_x, rect_y, linestyle="dashed", color="gray", dashes=(4, 4))

def dibujar_abanico(x_origin, y_origin, radius, lines_count=5):
    # Arco punteado
    theta = np.linspace(0, np.pi/2, 100)
    ax.plot(x_origin + radius * np.cos(theta), y_origin + radius * np.sin(theta),
            color="gray", linestyle="dashed", dashes=(4,4))

```

```

# Líneas radiales
for i in range(lines_count):
    angle = (np.pi/2) * i / (lines_count - 1)
    x_end = x_origin + radius * np.cos(angle)
    y_end = y_origin + radius * np.sin(angle)

    if i == 0 or i == lines_count-1:
        linewidth = 2.0 if i == 0 else 1.5
        ax.plot([x_origin, x_end], [y_origin, y_end], color="black", linewidth=linewidth)
    else:
        ax.plot([x_origin, x_end], [y_origin, y_end],
                color="gray", linestyle="dashed", dashes=(4,4), linewidth=0.8)

# Posición de los abanicos
dibujar_abanico(1, 0, 2.5, lines_count=5)
dibujar_abanico(3.75, 0, 2.5, lines_count=5)

# Ajustar la posición de la flecha y texto "Plumilla"
ax.annotate("Plumilla", xy=(1.7, 1.7), xytext=(0.5, 1.5),
            arrowprops=dict(facecolor="black", arrowstyle="->", linewidth=1),
            fontsize=10, ha="right", va="center")

# Ajustes de gráficos
ax.set_aspect("equal")
ax.axis("off")
plt.xlim(0, 7.5)
plt.ylim(-0.6, 3.5)

plt.tight_layout()
plt.savefig("diagrama_plumilla.png", dpi=150, bbox_inches="tight")
plt.close()
'

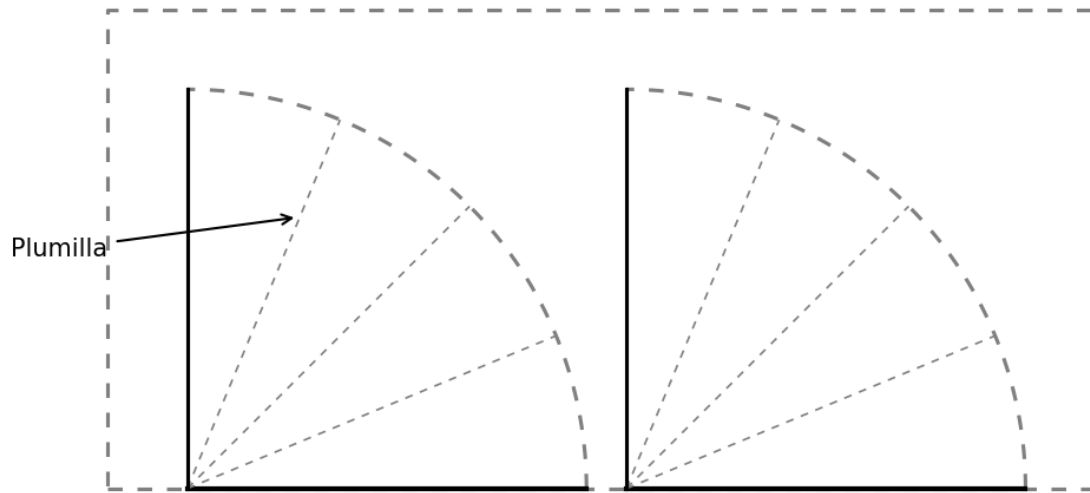
# Ejecutar código Python para generar el diagrama de plumillas
py_run_string(codigo_diagrama_plumilla)

```

Question

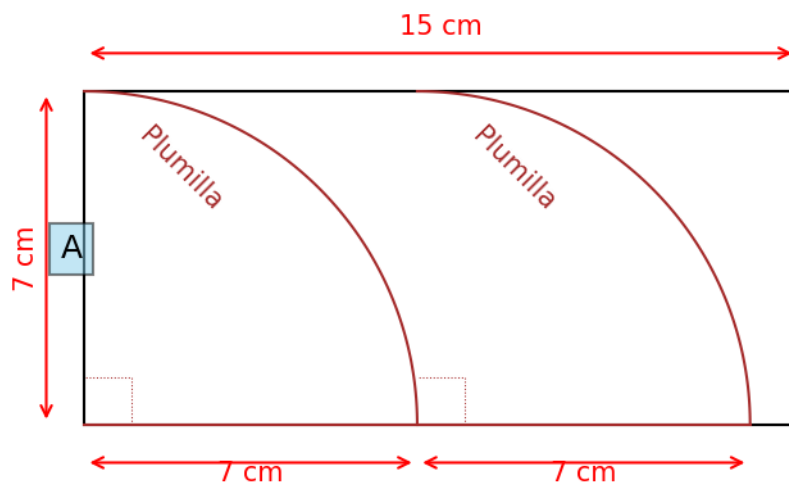
Las dimensiones de un parabrisas son 15 cm de largo y 7 cm de ancho. Este es un parabrisas plano y con forma de rectángulo, correspondiente a un modelo a escala.

El modelo a escala cuenta con 2 plumilla, limpiador, limpiaparabrisas, escobillas de 7 cm de longitud cada una para limpiar el parabrisas. Estas tienen un ángulo de apertura de ángulo recto, tal como se muestra a continuación.

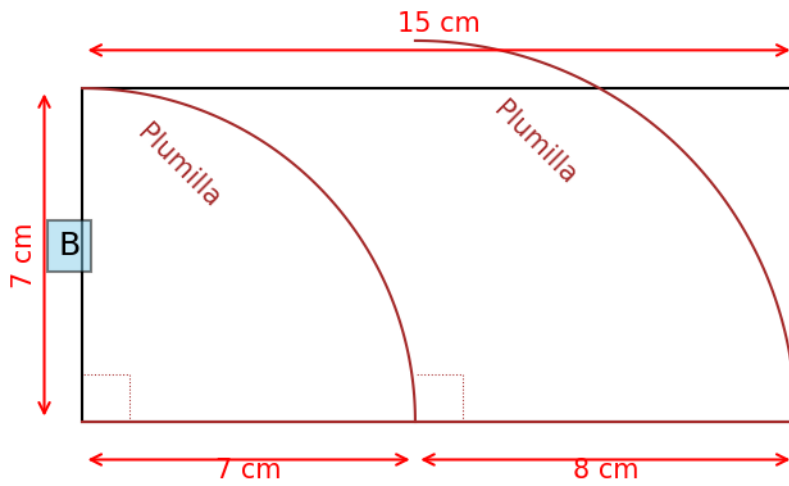


El esquema que representa los datos de las dimensiones del parabrisas es:

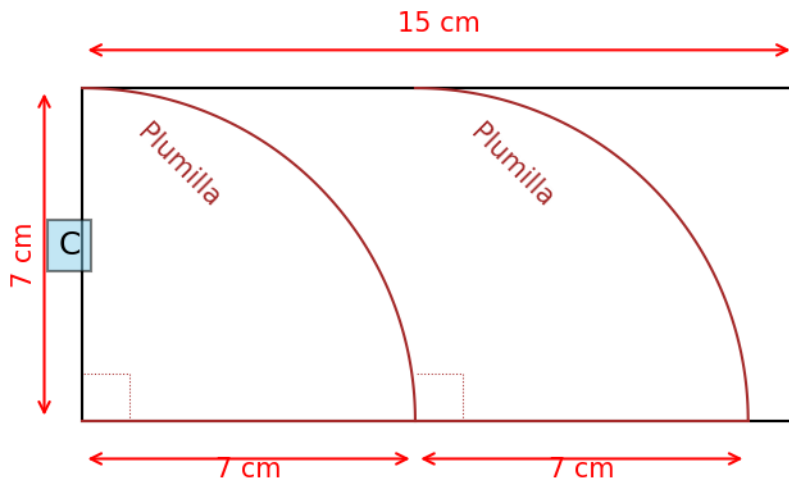
Answerlist



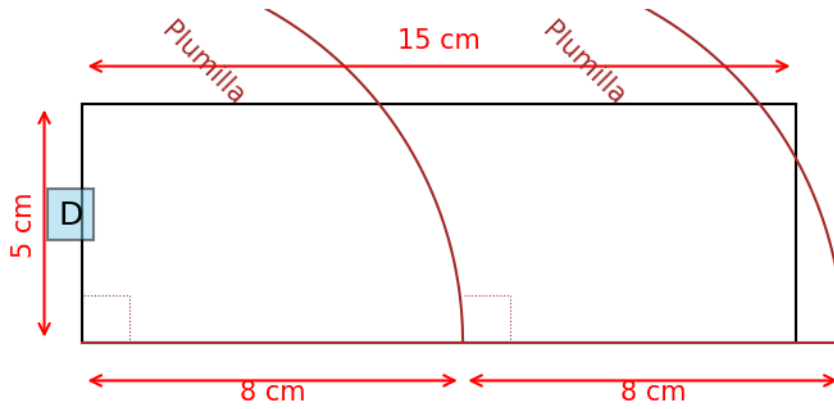
•



•



•



•

Solution

El esquema correcto es la opción C.

Para verificar cuál es el esquema correcto, debemos comparar las especificaciones dadas con cada una de las opciones:

- El parabrisas tiene 15 cm de largo y 7 cm de ancho.
- Cuenta con 2 plumilla, limpiador, limpiaparabrisas, escobillas de 7 cm de longitud cada una.
- Las plumilla, limpiador, limpiaparabrisas, escobillas tienen un ángulo de apertura de ángulo recto.

La opción C muestra correctamente: - Un rectángulo de 15 cm $\times$ 7 cm. - Dos plumilla, limpiador, limpiaparabrisas, escobillas con longitud de 7 cm cada una. - Ambas plumilla, limpiador, limpiaparabrisas, escobillas tienen un ángulo de barrido de 90°.

Las otras opciones presentan inconsistencias con los datos proporcionados:

- Opción A: Las dimensiones de las plumillas no coinciden con los datos del problema.
- Opción B: La segunda plumilla tiene 8 cm de longitud, lo cual no coincide con los datos del problema.
- Opción D: El ancho del parabrisas es 5 cm en lugar de 7 cm, y las plumillas miden 8 cm en lugar de 7 cm.

Answerlist

- Falso
- Falso
- Verdadero
- Falso

Meta-information

exname: parabrisas_plumillas_juguete extype: schoice exsolution: 0010 exshuffle: TRUE exsection: Geometría